UNITÀ 7 · IL SISTEMA OPERATIVO COME GESTORE DI RISORSE

# L'allocazione non contigua a paginazione statica

Basta buchi: tagliamo i programmi a fette tutte uguali e le mettiamo dove capita. Una tabella e due conti ci dicono sempre dove sono.

PER COMINCIARE

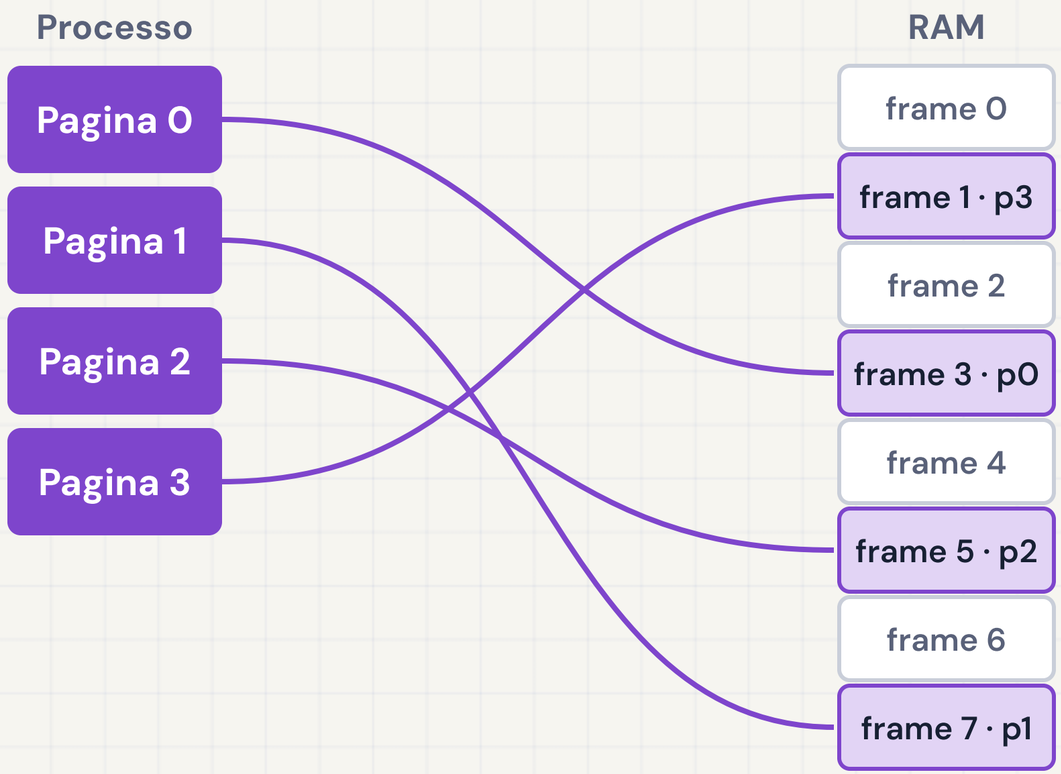
# Dopo la lezione sui buchi: quale idea ti convince di più?

SONDAGGIO

- **A** Compattare la RAM più spesso
- **B** Comprare più RAM e non pensarci
- **C** Tagliare i programmi in pezzi tutti uguali
- **D** Fare tante partizioni piccole e fisse

L'IDEA

# Pagine del processo, frame della RAM




Il processo è diviso in **pagine**, la RAM in **frame**: stessa dimensione (per esempio 4 KB).

Ogni pagina va in un frame libero qualsiasi, anche non vicino agli altri.

Statica: all'avvio tutte le pagine vengono caricate in RAM.

LA TRADUZIONE

# Due conti e una tabella

numero di pagina e offset (spiazzamento)

```
p = logico DIV dim
d = logico MOD dim
```

indirizzo fisico

```
fisico = frame(p) × dim + d
```

| Pagina | Frame |
|---|---|
| 0 | 3 |
| 1 | 7 |
| 2 | 5 |
| 3 | 1 |

La **tabella delle pagine**: una per ogni processo, la consulta la MMU.

ESEMPIO SVOLTO

# Pagine da 1024 byte, indirizzo logico 3000

**1**

**Pagina**

3000 DIV 1024 = **2**

**2**

**Offset**

3000 - 2048 = **952**

**3**

**Tabella**

pagina 2 → frame **5**

**4**

**Fisico**

5 × 1024 + 952 = **6072**

Controllo veloce: l'offset deve essere sempre minore di 1024, e il fisico deve cadere dentro il frame 5, cioè tra 5120 e 6143.

METTITI ALLA PROVA

# Stessa tabella, pagine da 1024 byte. Indirizzo logico 1500: qual è il fisico?

QUIZ

A 8668

B 3548

C 7000

D 7644

AL COMPUTER · 15 minuti

# Una MMU a pagine in Calc

1. In F2:F5 scrivi le pagine 0, 1, 2, 3 e in G2:G5 i frame 3, 7, 5, 1.
2. In A2:A6 gli indirizzi logici 0, 1500, 3000, 4095, 2048.
3. B2 `=QUOZIENTE(A2;1024)`, C2 `=RESTO(A2;1024)`, D2 `=CERCA.VERT(B2;$F$2:$G$5;2;0)`, E2 `=D2*1024+C2`. Copia in basso.
4. Prova anche 5000. Salva come **mmu-pagine.ods**.

**Controllo:** in E2:E6 leggi 3072, 7644, 6072, 2047, 5120. Con 5000 compare un errore (#N/D): la pagina 4 non esiste nella tabella.

IL TRUCCO DELL'INFORMATICO

# Con pagine da $2^k$ byte non serve dividere

Pagine da 4 KB = $2^{12}$ byte: gli ultimi **12 bit** sono l'offset, gli altri il numero di pagina.

12 bit = 3 cifre esadecimali. Basta guardare l'indirizzo!

Tabella: pagina 2 → frame 5.

logico

`0x2ABC`

fisico: cambia solo la pagina

`0x5ABC`

QUIZ A TRABOCCHETTO

# Pagine da 4 KB. La tabella dice: pagina 0x1F → frame 0x8. Indirizzo logico 0x1F3A4?

QUIZ

A 0x8F3A4

B 0x3A48

C 0x83A4

D 0x1F8

AL COMPUTER · 10 minuti

# La MMU nella console del browser

Premi **F12**, scheda **Console**. Scrivi una riga alla volta e premi Invio:

```
// pagina: sposta di 12 bit a destra
0x2ABC >> 12
// offset: tieni gli ultimi 12 bit
(0x2ABC & 0xFFF).toString(16)
// fisico: frame 5 al posto della pagina
((5 << 12) | 0xABC).toString(16)
```

Poi rifai il quiz: `((8 << 12) | (0x1F3A4 & 0xFFF)).toString(16)`

**Controllo:** le tre righe danno **2**, **"abc"** e **"5abc"**. L'ultima dà **"83a4"**. Copia i risultati in un file **console-pagine.txt**.

MENTI CHE CI HANNO PRECEDUTO

# Un hobby che aveva bisogno delle pagine

> **Sto facendo un sistema operativo (libero) (solo un hobby, non sarà grande e professionale come gnu) per cloni AT 386(486).**
>
> **Linus Torvalds**, studente all'Università di Helsinki, messaggio sul newsgroup comp.os.minix, 25 agosto 1991

Perché proprio il 386? Fu il primo processore x86 con la **paginazione** in hardware, a pagine da 4 KB. L'idea però era nata nel 1962 a Manchester, sul computer **Atlas**.

QUANTO COSTA?

# Anche la tabella delle pagine occupa memoria

indirizzi a 32 bit, pagine da 4 KB (12 bit di offset)

$2^{32-12} = 2^{20}$ pagine

$2^{20} \times 4$ byte = 4 MB

per ogni processo, se la tabella fosse un'unica lista

- Per questo si usano tabelle **a più livelli**: si crea solo la parte che serve
- La **TLB** è una piccola cache dentro la MMU con le traduzioni recenti



FRAMMENTAZIONE

# Niente buchi fuori, un po' di spreco dentro

### Esterna: sparita

Ogni frame libero va bene per qualunque pagina. Se ci sono N frame liberi, entra qualunque processo di N pagine.

### Interna: solo nell'ultima pagina

Processo di 10 000 byte, pagine da 4096: servono **3 pagine** (12 288 byte), si sprecano **2288 byte**.

> Limite della paginazione **statica**: tutto il processo deve stare in RAM. Un programma più grande della RAM non parte. Alla prossima lezione lo risolviamo.

PUZZLE

# Metti in ordine la traduzione di un indirizzo

1. Cerca la pagina nella TLB o nella tabella delle pagine
2. La MMU lo divide in numero di pagina e offset
3. Unisce frame e offset: indirizzo fisico
4. La CPU genera un indirizzo logico
5. La RAM restituisce il dato
6. Ottiene il numero di frame

SIMULAZIONE IN JAVA

# Il traduttore di indirizzi

```
import java.util.Scanner;
public class Traduttore {
  public static void main(String[] args) {
    final int DIM = 1024;
    int[] tabella = {3, 7, 5, 1};            // pagina -> frame
    Scanner in = new Scanner(System.in);
    System.out.print("Indirizzo logico: ");
    int logico = in.nextInt();
    int pagina = logico / DIM;
    int offset = logico % DIM;
    if (pagina >= tabella.length) { System.out.println("Errore: pagina inesistente"); return; }
    int fisico = tabella[pagina] * DIM + offset;
    System.out.println("p=" + pagina + " d=" + offset + " fisico=" + fisico);
  }
}
```

AL COMPUTER · 20 minuti

# Prova il traduttore su OnlineGDB

1. Su **onlinegdb.com** scegli **Java**, copia il programma e premi **Run**.
2. Prova 3000, 1500, 0 e 4096. Confronta con il foglio Calc.
3. Cambia DIM in 4096 e prova 10940 (è 0x2ABC): ritrovi il risultato della console?
4. Aggiungi una quinta pagina nel frame 0 e prova di nuovo 4096.
5. Scarica il file come **Traduttore.java**.

**Controllo:** 3000 → p=2 d=952 fisico=6072; 1500 → 7644; 0 → 3072; 4096 → errore. Con DIM 4096, 10940 → p=2 d=2748 fisico=23228, cioè 0x5ABC. Con la quinta pagina, 4096 → fisico=0.

# La stessa idea, cinquant'anni dopo, dentro i chatbot

Mentre risponde, un modello linguistico tiene in memoria della GPU i calcoli già fatti su ogni conversazione.

Riservare un blocco unico per conversazione creava frammentazione. La soluzione è stata dividere quella memoria in **blocchi** uguali, con una **tabella dei blocchi**.



SFIDA A SQUADRE

# Caccia all'indirizzo: apri il lucchetto



- 1ª cifra: il numero di pagina dell'indirizzo logico 3500 (pagine da 1024)
- 2ª cifra: il frame che contiene l'indirizzo logico 2100
- 3ª cifra: quante cifre esadecimali di offset con pagine da 4 KB
- 4ª cifra: l'ultima cifra dell'indirizzo fisico del logico 3000

```
codice
```

PUZZLE

# Abbina ogni termine al suo significato

| | | |
|---|---|---|
| **Pagina** | --- | Cache delle traduzioni più recenti |
| **Frame** | --- | Fetta di dimensione fissa del processo |
| **Tabella delle pagine** | --- | Fetta della RAM grande quanto una pagina |
| **Offset** | --- | Dice in quale frame sta ogni pagina |
| **TLB** | --- | Posizione del byte dentro la pagina |

BIGLIETTO D'USCITA

# Completa prima di uscire

Le fette della RAM grandi quanto una pagina si chiamano _____.

Con pagine da 1024 byte, l'indirizzo logico 2050 è nella pagina ____ con offset ____.

Con pagine da 4 KB l'offset occupa ____ bit.

La paginazione elimina la frammentazione _____.

⏱ 3:00

RIEPILOGO

# Cosa portiamo a casa

- Pagine del processo in frame qualsiasi
- p = logico DIV dim, d = logico MOD dim
- fisico = frame × dim + d
- Pagine da $2^k$: l'offset sono gli ultimi k bit
- Niente frammentazione esterna
- Statica: tutto il processo in RAM

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.